



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ N 3

Название: «Программная реализация комплексной криптосистемы на базе RSA»

Дисциплина: «Защита информации»

Студент ИУ7-72Б
(Группа)

(Подпись, дата)

Ю.А. Попов
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю.С. Руденкова
(И. О. Фамилия)

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Анализ предметной области	4
1.1 Алгоритм RSA	4
1.2 Гибридное шифрование, контроль целостности и подпись	4
1.3 Защита закрытого ключа	5
2 Проектирование и реализация программы	6
2.1 Структура программы и форматы данных	6
2.2 Генерация и использование ключей	6
2.3 Операции с файлами	7
3 Тестирование	9
3.1 Методика тестирования	9
3.2 Результаты тестирования	9
ЗАКЛЮЧЕНИЕ	11
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	12

ВВЕДЕНИЕ

Асимметричная криптография позволяет разделить ключи, предназначенные для шифрования и расшифрования. Это упрощает передачу защищаемой информации, но модульные операции RSA значительно медленнее симметричных преобразований. Поэтому для файлов практична гибридная схема: RSA защищает короткий случайный сеансовый ключ, а содержимое шифруется алгоритмом AES.

Цель работы — разработать на языке Python комплексную криптосистему на базе RSA, включающую гибридное шифрование файлов, электронную подпись и защиту закрытого ключа мастер-паролем.

Для достижения цели необходимо решить следующие задачи:

- 1) реализовать генерацию пары ключей RSA с проверкой простоты кандидатов;
- 2) реализовать гибридное шифрование данных произвольного объема с инкапсуляцией ключа AES посредством RSA;
- 3) реализовать создание и верификацию электронной подписи SHA-256/RSA;
- 4) защитить закрытый ключ при записи на диск с помощью PBKDF2;
- 5) провести граничное тестирование и проверку реакции на изменение данных.

1 Анализ предметной области

1.1 Алгоритм RSA

RSA использует модуль n , который образуется произведением двух больших простых чисел p и q . Открытый ключ состоит из пары (n, e) , а закрытый ключ включает показатель d . В стандарте PKCS N 1 определены примитивы RSA, схемы шифрования, подписи и форматы представления ключей [1].

При генерации ключей вычисляются значения

$$n = p \cdot q, \quad \varphi(n) = (p - 1)(q - 1), \quad de \equiv 1 \pmod{\varphi(n)}. \quad (1.1)$$

В реализации открытый показатель выбирается равным 65537. Перед вычислением d проверяется условие взаимной простоты $\gcd(e, \varphi(n)) = 1$. Вероятностный тест Миллера–Рабина применяется к каждому кандидату на p и q . Кандидат разлагается как $a \cdot 2^s + 1$, после чего для случайных баз проверяется отсутствие нетривиальных свидетелей составности.

Прямое возведение сообщения в степень по модулю не обеспечивает семантическую безопасность. Поэтому в работе сеансовый ключ кодируется по схеме RSAES-OAEP с SHA-256 и функцией генерации маски MGF1. Эта схема описана в PKCS N 1 вместе с ограничением на длину сообщения относительно размера модуля [1].

1.2 Гибридное шифрование, контроль целостности и подпись

AES является симметричным блочным алгоритмом, стандартизованным в FIPS 197; стандарт определяет варианты ключей длиной 128, 192 и 256 бит [2]. В программе используется AES-256 в режиме счетчика CTR. Для каждого файла создается случайный 256-битный ключ K и 16-байтовое начальное значение счетчика N . Преобразование i -го блока можно представить в виде

$$C_i = P_i \oplus \text{AES}_{K(N+i)}, \quad (1.2)$$

где P_i — блок открытого текста; C_i — блок шифротекста; N — начальное значение счетчика. Повторное применение этого же преобразования восстанавливает P_i .

Режим CTR обеспечивает конфиденциальность, но сам не обнаруживает изменение шифротекста. Поэтому контейнер содержит тег HMAC-SHA-256 от контекста, счетчика и шифротекста. SHA-256 относится к функциям семейства Secure Hash Standard [3]. Перед расшифрованием тег сравнивается функцией постоянного времени; при несовпадении открытый текст не записывается.

Электронная подпись создается от хэша исходного файла. В реализации применено кодирование EMSA-PKCS1-v1_5 с идентификатором SHA-256 и возведение полученного блока в степень d по модулю n . При проверке подпись возводится в степень e и сравнивается с вновь сформированным блоком хэша. Описание RSASSA-PKCS1-v1_5 приведено в PKCS N 1 [1].

1.3 Защита закрытого ключа

Закрытый ключ не записывается в открытом виде. Из мастер-пароля и случайной 16-байтовой соли выводится 32-байтовый ключ функцией PBKDF2-HMAC-SHA256. В библиотеке Python эта функция доступна как `hashlib.pbkdf2_hmac` и принимает имя хэш-функции, пароль, соль, число итераций и желаемую длину ключа [4]. В программе установлено 310000 итераций. Полученным ключом шифруется JSON представление закрытого ключа алгоритмом AES-256-CTR, а HMAC-SHA-256 защищает его от подмены.

2 Проектирование и реализация программы

2.1 Структура программы и форматы данных

Исходный код расположен в каталоге `src`. Файл `rsa_crypto.py` содержит математические операции RSA, AES-256, функции сохранения ключей и операции с файлами. Файл `main.py` реализует консольные команды. Состав основных компонентов приведен в Таблица 2.1.

Таблица 2.1 — Основные компоненты программы

Компонент	Основные элементы	Назначение
<code>rsa_crypto.py</code>	<code>is_probable_prime</code> , <code>generate_key_pair</code>	Проверка простоты Миллера–Рабина и генерация ключей RSA.
<code>rsa_crypto.py</code>	<code>rsa_oaep_encrypt</code> , <code>rsa_oaep_decrypt</code>	ОАЕР-инкапсуляция 32-байтового сеансового ключа.
<code>rsa_crypto.py</code>	<code>aes256_ctr</code> , <code>_authenticated_encrypt</code>	Шифрование AES-256-CTR и проверка HMAC-SHA-256.
<code>rsa_crypto.py</code>	<code>sign</code> , <code>verify</code>	Создание и проверка подписи SHA-256/RSA.
<code>rsa_crypto.py</code>	<code>save_private_key</code> , <code>load_private_key</code>	PBKDF2-защита закрытого ключа при сохранении.
<code>main.py</code>	<code>keygen</code> , <code>encrypt</code> , <code>decrypt</code> , <code>sign</code> , <code>verify</code>	Интерфейс командной строки.

Открытый ключ хранится в JSON-контейнере формата `lab03-public-v1` и содержит n и e . Закрытый ключ хранится только в контейнере `lab03-private-v1`; в нем расположены соль, число итераций PBKDF2, счетчик, шифротекст и тег HMAC. Зашифрованный файл имеет формат `lab03-envelope-v1` с полями `encrypted_key`, `nonce`, `ciphertext` и `tag`. Бинарные значения переводятся в Base64, что позволяет представлять контейнеры в JSON.

2.2 Генерация и использование ключей

Размер модуля по умолчанию составляет 2048 бит. Допускается размер 1024 бита только для ускорения автоматических тестов. Для криптографически случайных значений применяются `secrets.randbits` и `secrets.token_bytes`, а не генератор псевдослучайных чисел общего назначения.

Фрагмент генерации кандидата и проверки простоты приведен в Листинг 2.1.

```
1 def _generate_prime(bits: int) -> int:
2     while True:
3         value = secrets.randbits(bits) | (1 << (bits - 1)) | 1
4         if is_probable_prime(value):
5             return value
```

Листинг 2.1 — Генерация нечетного кандидата заданной битовой длины

Алгоритм исключает числа, делящиеся на малые простые, затем выполняет 40 раундов Миллера–Рабина. После нахождения различных p и q программа убеждается, что e взаимно просто с $\varphi(n)$, и вычисляет d как обратный элемент по модулю $\varphi(n)$.

2.3 Операции с файлами

При шифровании формируется случайный сеансовый ключ. Он передается в `rsa_oaep_encrypt`, а все байты входного файла обрабатываются AES-256-CTR. Получившиеся шифротекст и HMAC вместе с RSA-зашифрованным сеансовым ключом сохраняются в одном контейнере. Таким образом, RSA применяется только к 32 байтам ключевого материала, а длина файла не ограничивается размером модуля.

Порядок операций показан в листинге 2.2.

```
1 исходный файл --> AES-256-CTR --> ciphertext + HMAC --> JSON-контейнер
2
3
4 случайный ключ K --> RSAES-OAEP --> encrypted_key
```

Листинг 2.2 — Формирование гибридного криптографического контейнера

Команда `decrypt` сначала восстанавливает сеансовый ключ закрытым RSA-ключом, после чего проверяет HMAC. Лишь успешно прошедший проверку шифротекст расшифровывается и записывается в выходной файл. Аналогично, команды `sign` и `verify` работают с отдельным файлом подписи формата `lab03-signature-v1`.

Пользовательский интерфейс запускается из корня проекта. Основные команды приведены в Листинг 2.3.

```
1 python3 src/main.py keygen public.json private.json
2 python3 src/main.py encrypt public.json input.bin encrypted.json
3 python3 src/main.py decrypt private.json encrypted.json restored.bin
4 python3 src/main.py sign private.json input.bin input.sig
5 python3 src/main.py verify public.json input.bin input.sig
```

Листинг 2.3 — Команды консольного интерфейса

3 Тестирование

3.1 Методика тестирования

Автоматические тесты расположены в `tests/test_rsa_crypto.py` и запускаются из корня проекта командой `python3 -m unittest discover -s tests -v`. В тестах однократно создается 1024-битная пара RSA-ключей, после чего для каждого сценария выполняются шифрование, расшифрование, создание подписи и ее проверка. Набор проверок приведен в Таблица 3.1.

Таблица 3.1 — Набор автоматических тестов

N	Проверяемое условие	Ожидаемый результат
1	Тестовый вектор AES-256 для одного блока.	Полученный блок совпадает с опубликованным контрольным значением.
2	Сохранение и загрузка закрытого ключа с правильным мастер-паролем.	Восстановленный ключ совпадает с исходным.
3	Загрузка закрытого ключа с неверным паролем.	Возвращается ошибка проверки тега.
4	Пустой файл.	Расшифрованный файл пуст; подпись корректна.
5	Файл длиной один байт.	Расшифрованный файл побайтно равен исходному; подпись корректна.
6	Файл длиной 5120 байт, превышающей размер блока RSA.	Файл восстанавливается полностью; подпись корректна.
7	Изменение контейнера или исходного файла после подписи.	Расшифрование или проверка подписи завершается ошибкой.

Сценарий с файлом длиной 5120 байт принципиален: он подтверждает, что содержимое не передается в RSA напрямую. В RSA-OAEP защищается только 32-байтовый AES-ключ, поэтому размер данных определяется возможностями симметричного преобразования и формата контейнера.

3.2 Результаты тестирования

Выполненная команда тестирования имеет вид:

```
1 python3 -m unittest discover -s tests -v
```

Листинг 3.1 — Запуск автоматических тестов

Получен следующий результат: четыре тестовых метода выполнены успешно без ошибок. В их составе проверены контрольный вектор AES, защита ключа мастер-паролем, три граничных размера файлов и реакция на преднамеренное изменение шифротекста и подписанного файла. Следовательно, система обеспечивает обратимое шифрование для проверенных объемов данных, контроль целостности и аутентичности исходного файла.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы создана программа на Python, реализующая комплексную криптосистему на базе RSA. При генерации ключей используются случайные простые числа и тест Миллера–Рабина. Для исключения ограничений RSA при обработке больших файлов реализована гибридная схема: AES-256-CTR шифрует содержимое, а RSAES-OAEP защищает случайный сеансовый ключ.

В программе реализованы электронная подпись SHA-256/RSA, проверка целостности HMAC-SHA-256 и защищенное хранение закрытого ключа. Ключ шифруется ключом, полученным из мастер-пароля PBKDF2-HMAC-SHA256, с индивидуальной случайной солью. Автоматические тесты успешно подтвердили работоспособность на пустом, однобайтном и большом файлах, а также корректное обнаружение неверного пароля и модификации данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Moriarty K.; Kaliski B.; Jonsson J.; Rusch A. PKCS N 1: RSA Cryptography Specifications Version 2.2 [Электронный ресурс]. — 2016. — Режим доступа: <https://www.rfc-editor.org/rfc/rfc8017.html>.
2. Technology. FIPS 197: Advanced Encryption Standard (AES) [Электронный ресурс]. — 2023. — Режим доступа: <https://csrc.nist.gov/pubs/fips/197/final>.
3. Technology. FIPS 180-4: Secure Hash Standard [Электронный ресурс]. — 2015. — Режим доступа: <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>.
4. Python Software Foundation. hashlib: Secure hashes and message digests [Электронный ресурс]. — 2026. — Режим доступа: https://docs.python.org/3/library/hashlib.html#hashlib.pbkdf2_hmac.